

Control de posición de motor DC

Evaluación Globalizadora A - Informe final

Lucas Ignacio Gonzalez
lucasignaciogonzalez1999@gmail.com

Fernando Jorja
quilmesmakeup@hotmail.com

Universidad Tecnológica Nacional - Facultad Regional Avellaneda
Técnicas Digitales III - Curso 511

Abstract— El presente informe describe el desarrollo de un sistema embebido para el control de posición angular de un motor de corriente continua. El proyecto utiliza un microcontrolador ESP32-S3 como unidad de procesamiento, un encoder magnético AS5600 para la medición de posición, un driver L298N para el accionamiento del motor y un firmware basado en ESP-IDF con FreeRTOS. El sistema propuesto permite ingresar un ángulo objetivo, medir la posición real del eje, calcular el error angular y accionar el motor mediante PWM y señales de dirección. Además, se prevé la implementación de un controlador PID, perfiles de movimiento, configuración local mediante pulsadores y display LCD I2C, configuración remota mediante UART, almacenamiento en memoria no volátil y detección de bloqueos durante el movimiento.

Palabras Clave— FreeRTOS, ESP32-S3, PID, motor DC, AS5600, PWM, I2C, UART, L298N, NVS.

ÍNDICE

I. Introducción	1
II. Fundamentación	1
A. Problemática	1
B. Solución técnica	1
III. Objetivos y alcance	1
A. Alcance comprobado y estado del desarrollo	2
IV. Diagrama en bloques	2
V. Hardware seleccionado	2
A. ESP32-S3	2
B. Encoder AS5600	2
C. Driver L298N y motor DC	2
D. Display LCD, pulsadores y UART	2
E. Pines propuestos	2
VI. Esquemático, PCB y test points	2
A. Test points	2
VII. Control PID y error angular	2
VIII. Arquitectura FreeRTOS	2
IX. Comunicación, interrupciones y persistencia	2
A. Colas	3
B. Interrupciones y semáforos	3
C. Persistencia	3
X. Detección de bloqueo y perfiles de movimiento	3
A. Detección de bloqueo	3
B. Perfiles de movimiento	3
XI. Manual de usuario	3
A. Encendido	3
B. Uso local	3
C. Comandos UART	3
XII. Conclusiones	3
XIII. Referencias	3
Anexo I - Diagrama de tareas	4
Anexo II - Esquemático	4
Anexo III - PCB	5

I. INTRODUCCIÓN

El proyecto consiste en implementar un control de posición para un motor de corriente continua. En un accionamiento de lazo abierto, el motor únicamente recibe una tensión o PWM y gira con una velocidad dependiente de la alimentación, la carga y el rozamiento. En cambio, para ubicar el eje en una posición específica se necesita medir la posición real, compararla con la posición deseada y corregir el movimiento de forma automática.

La solución desarrollada cierra el lazo mediante un encoder magnético AS5600 acoplado al eje del motor por medio de un

imán. El ESP32-S3 lee el ángulo por I2C, calcula el error angular normalizado entre -180° y $+180^\circ$, ejecuta un controlador PID y aplica la salida al L298N. El signo de la salida define el sentido de giro y su módulo define el duty cycle de PWM.

El firmware fue organizado con FreeRTOS para separar las responsabilidades del sistema. Las tareas de control y adquisición se ejecutan con mayor prioridad, mientras que las tareas de interfaz, display y almacenamiento se ejecutan con prioridad menor y se bloquean cuando no tienen trabajo pendiente.

II. FUNDAMENTACIÓN

A. Problemática

Un motor DC convencional no posee una referencia absoluta de posición. Si se lo alimenta directamente, el eje gira hasta que se quite la alimentación o hasta que una carga mecánica lo detenga. Esta forma de operación no permite seleccionar un ángulo, repetir posiciones ni corregir desviaciones.

Además, el sistema trabaja con ángulos de 0° a 360° . Por lo tanto, el error no debe calcularse como una resta lineal simple. Por ejemplo, para ir de 350° a 10° el desplazamiento correcto por camino mínimo es $+20^\circ$, no -340° . Esta particularidad obliga a normalizar el error antes de aplicarlo al PID.

B. Solución técnica

El lazo de control queda formado por el motor, el imán acoplado al eje, el encoder AS5600, el ESP32-S3 y el puente H L298N. El AS5600 entrega la posición absoluta del eje con resolución interna de 12 bits. El ESP32-S3 convierte la lectura cruda a grados, calcula el error por camino corto y genera la acción de control. El L298N recibe ENA por PWM e IN1/IN2 como señales de sentido de giro.

Para la interfaz se utilizan cuatro pulsadores y un display LCD 16x2 con expansor PCF8574. La configuración remota se realiza por UART TTL de 3,3 V. La persistencia de setpoint, ganancias PID y perfil de movimiento se resuelve mediante NVS en la memoria flash del ESP32-S3.

III. OBJETIVOS Y ALCANCE

El objetivo principal es controlar la posición angular de un motor DC mediante un sistema embebido con RTOS, lazo cerrado y accionamiento PWM.

- Ingresar y modificar el ángulo deseado desde interfaz local y UART.
- Medir la posición actual del eje con el AS5600.
- Calcular el error angular por el camino de menor desplazamiento.
- Implementar un controlador PID ajustable.
- Accionar el motor con PWM y señales de dirección sobre el L298N.
- Implementar dos perfiles de movimiento: escalón y rampa.
- Detectar bloqueo mecánico y solicitar recuperación por cambio de sentido.
- Guardar configuración en memoria no volátil y recuperarla al reiniciar.
- Utilizar tareas, colas, semáforos e interrupciones reales de hardware.

A. Alcance comprobado y estado del desarrollo

El firmware contiene la arquitectura completa de tareas, colas, semáforos, UART, NVS, control PID, manejo de display, lectura del AS5600, accionamiento del motor y supervisión de bloqueo. El PCB y el esquemático fueron diseñados en KiCad, y el sistema cumple con las condiciones principales solicitadas para la EGA.

IV. DIAGRAMA EN BLOQUES

La Fig. 1 muestra la interconexión general. El ESP32-S3 concentra el procesamiento; GPIO4 se utiliza para PWM hacia ENA del L298N, GPIO5 y GPIO6 para IN1/IN2, GPIO8/GPIO9 para SDA/SCL compartidos entre AS5600 y LCD, GPIO10 a GPIO13 para pulsadores y GPIO17/GPIO18 para UART.

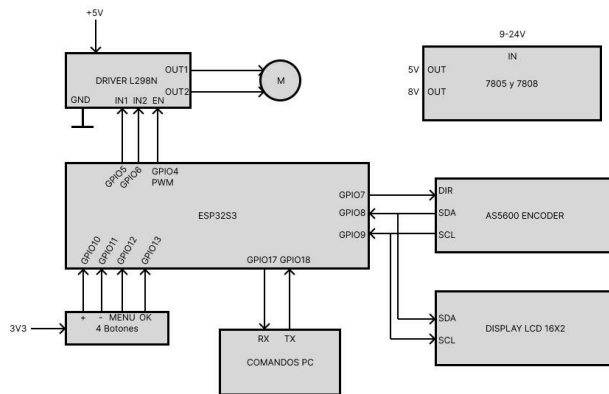


Fig. 1. Diagrama en bloques del sistema con interconexión de pines.

V. HARDWARE SELECCIONADO

A. ESP32-S3

El ESP32-S3 se utiliza como unidad central. Sus funciones son ejecutar FreeRTOS, leer el AS5600 por I2C, generar PWM mediante LEDC, manejar pines GPIO, recibir interrupciones de botones, comunicarse por UART y almacenar configuración en NVS.

B. Encoder AS5600

El AS5600 es un encoder magnético absoluto con lectura angular de 0° a 360° e interfaz I2C. Su resolución interna de 12 bits permite 4096 posiciones por vuelta, por lo que el paso angular teórico es $360^\circ/4096 = 0,0879^\circ$. La precisión práctica dependerá de la alineación del imán, la distancia al sensor y el montaje mecánico.

C. Driver L298N y motor DC

El L298N se utiliza como puente H. La entrada ENA recibe PWM desde el ESP32-S3, mientras que IN1 e IN2 determinan el sentido de giro. El motor DC opera en baja tensión y se acopla mecánicamente al imán del AS5600. Todas las masas del sistema deben estar unidas para que las referencias lógicas sean comunes.

D. Display LCD, pulsadores y UART

El display LCD 16x2 se maneja por I2C mediante PCF8574. Los pulsadores +, -, MENU y OK permiten navegar parámetros y confirmar movimientos. La UART trabaja a nivel TTL de 3,3 V; no debe conectarse una señal de 5 V directamente al ESP32-S3.

E. Pines propuestos

- GPIO17 y GPIO18: UART TX/RX hacia adaptador USB-UART TTL de 3,3 V.
- GPIO10, GPIO11, GPIO12 y GPIO13: pulsadores +, -, MENU y OK.
- GPIO9: I2C SCL para AS5600 y LCD.
- GPIO8: I2C SDA para AS5600 y LCD.

- GPIO7 o GND: selección DIR del AS5600.
- GPIO6: dirección motor 2 hacia IN2 del L298N.
- GPIO5: dirección motor 1 hacia IN1 del L298N.
- GPIO4: PWM motor hacia ENA del L298N.

VI. ESQUEMÁTICO, PCB Y TEST POINTS

El esquemático integra el ESP32-S3, conectores externos, display LCD, botones, encoder, reguladores de tensión, driver L298N y bornes de alimentación/motor. Por su tamaño y nivel de detalle, el esquemático completo se incluye como anexo a página completa, para evitar que pierda legibilidad dentro de una columna. El PCB también se presenta en anexo, donde puede observarse con mayor claridad el ruteo y el membrete del proyecto.

A. Test points

Los test points permiten verificar alimentación, masa y señales críticas durante la puesta en marcha. La lista debe contrastarse con el esquemático final antes de fabricar o entregar el PCB, especialmente si se reenumeran conectores en KiCad.

TP0: GND de referencia del ESP32-S3. TP1: GND de potencia. TP2: salida hacia motor / puente H. TP4: VCC 5 V para verificar alimentación de periféricos. TP5: alimentación de potencia, aproximadamente 8 V. Estos puntos permiten comprobar continuidad de masa, presencia de alimentación y conmutación del puente H durante las pruebas.

VII. CONTROL PID Y ERROR ANGULAR

La lectura cruda del AS5600 se convierte a grados mediante la relación:

$$\text{angulo} = \text{raw} * 360.0f / 4096.0f;$$

El error angular se normaliza para permanecer en el intervalo -180° a +180°. De esta forma, el sistema elige el desplazamiento de menor módulo.

$$\begin{aligned} \text{setpoint} = 10^\circ, \text{posición} = 350^\circ &\rightarrow \text{error} = +20^\circ \\ \text{setpoint} = 350^\circ, \text{posición} = 10^\circ &\rightarrow \text{error} = -20^\circ \end{aligned}$$

La ecuación conceptual del PID es:

$$\text{salida} = K_p * \text{error} + K_i * \text{integral} + K_d * \text{derivada};$$

En el firmware, la salida se limita y se transforma en una orden para el motor. Si la salida es positiva, el motor gira en un sentido; si es negativa, gira en sentido contrario. El módulo de la salida determina el PWM aplicado. También se incorporan antiwindup, boost de arranque para vencer rozamiento y control por pulsos en la zona fina para reducir oscilaciones cerca del objetivo.

VIII. ARQUITECTURA FREERTOS

El diagrama de tareas completo se presenta en el Anexo I para conservar su legibilidad. La arquitectura separa responsabilidades entre tareas de control, lectura de sensor, administración del bus I2C, interfaz local, comunicación UART, persistencia y seguridad. Las tareas de prioridad 3 atienden funciones críticas de control y bus; prioridad 2 se reserva para comandos remotos y seguridad; prioridad 1 queda asignada a interfaz y almacenamiento no crítico.

Tareas principales:

- PID_Task (prioridad 3) realiza control periódico, cálculo PID y accionamiento PWM/IN1/IN2;
- AS5600_Reader_Task (prioridad 3) lee periódicamente el encoder y publica el ángulo;
- I2C_Manager_Task (prioridad 3) arbitra el bus I2C compartido;
- Safety_Task (prioridad 2) supervisa movimiento y bloqueo;
- UART_Command_Task (prioridad 2) interpreta comandos externos;
- ButtonHandler_Task (prioridad 1) procesa antirrebote y menú;
- Display_Task (prioridad 1) actualiza el LCD;
- Storage_Task (prioridad 1) gestiona NVS.

IX. COMUNICACIÓN, INTERRUPTONES Y PERSISTENCIA

A. Colas

La comunicación se realiza con colas para desacoplar módulos. ControlQueue recibe comandos desde botones, UART, Storage y Safety hacia PID_Task. I2C_TXQueue e I2C_RXQueue centralizan el bus I2C. angleQueue conserva la última posición angular. MotorStateQueue permite que Safety_Task observe el estado del motor. DisplayQueue desacopla mensajes de pantalla. ConfigQueue concentra guardado y recuperación de configuración.

B. Interrupciones y semáforos

Los pulsadores generan interrupciones GPIO. La ISR no procesa el menú: únicamente aplica una verificación temporal de antirrebote y libera el semáforo correspondiente con xSemaphoreGiveFromISR(). Luego ButtonHandler_Task toma el semáforo, interpreta la acción y envía comandos por colas. Este procesamiento diferido mantiene la ISR corta y evita ejecutar lógica extensa en contexto de interrupción.

C. Persistencia

La configuración persistente se modela con una estructura que contiene setpoint, Kp, Ki, Kd y perfil de movimiento. Storage_Task es la única tarea que accede a NVS, por lo que evita accesos concurrentes a la memoria no volátil. Al iniciar, recupera la configuración y la informa a PID_Task y Display_Task.

```
typedef struct {
    float setpoint;
    float kp;
    float ki;
    float kd;
    movement_profile_t profile;
} system_config_t;
```

X. DETECCIÓN DE BLOQUEO Y PERFILES DE MOVIMIENTO

A. Detección de bloqueo

Safety_Task detecta bloqueo cuando el motor se encuentra en movimiento, el PWM aplicado supera un mínimo, el error angular continúa siendo significativo y la posición medida prácticamente no cambia durante una ventana temporal. En ese caso envía CMD_BLOCK_DETECTED a PID_Task. El PID no apaga directamente el sistema, sino que fuerza una recuperación cambiando el sentido para intentar alcanzar el setpoint por el camino alternativo.

B. Perfiles de movimiento

Se definieron dos perfiles: ESCALÓN y RAMPA.

XI. MANUAL DE USUARIO

A. Encendido

- Conectar alimentación del motor y alimentación lógica respetando polaridad.
- Verificar GND común entre ESP32-S3, L298N, AS5600, LCD y adaptador UART.
- Al iniciar, el sistema recupera la última configuración guardada en NVS.
- El display muestra el menú inicial o el último setpoint guardado.

B. Uso local

- MENU recorre las opciones: ángulo, Kp, Ki, Kd y perfil.
- + y - modifican el valor mostrado.
- En el menú ángulo, OK confirma el setpoint y comienza el movimiento.
- En los menús Kp, Ki, Kd y perfil, los cambios se aplican al PID y se guardan.

C. Comandos UART

La UART se configura a 115200 baudios, 8 bits de datos, sin paridad y 1 bit de stop. Comandos previstos:

```
SET ANGLE 90
SET KP 5.0
SET KI 0.01
SET KD 0.2
SET PROFILE ESCALON
SET PROFILE RAMPA
SAVE
STOP
```

XII. CONCLUSIONES

Se desarrolló una solución de control de posición angular para motor DC basada en ESP32-S3, AS5600, L298N y FreeRTOS. La arquitectura cumple con la separación de responsabilidades pedida para la EGA, utiliza más de tres tareas con prioridades diferenciadas, colas para comunicación, semáforos liberados desde interrupciones GPIO, persistencia en NVS, comandos UART, HMI local y mecanismo de detección/recuperación ante bloqueo.

La utilización de una tarea guardiana para el bus I2C evita accesos simultáneos entre el sensor y el display. El cálculo de error angular por camino mínimo resuelve correctamente los cruces de 0°/360°. La división del firmware en tareas facilita la prueba por módulos y la defensa oral del comportamiento del sistema.

XIII. REFERENCIAS

- [1] FreeRTOS, “Mastering the FreeRTOS Real Time Kernel”.
- [2] Espressif Systems, “ESP-IDF Programming Guide”.
- [3] Espressif Systems, “ESP32-S3 Technical Reference Manual”.
- [4] ams OSRAM, “AS5600 12-bit Programmable Contactless Potentiometer Datasheet”.
- [5] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet”.
- [6] Apuntes de Técnicas Digitales III, UTN FRA: tareas, colas, interrupciones, administración de recursos y memoria en FreeRTOS.

ANEXO I - DIAGRAMA DE TAREAS

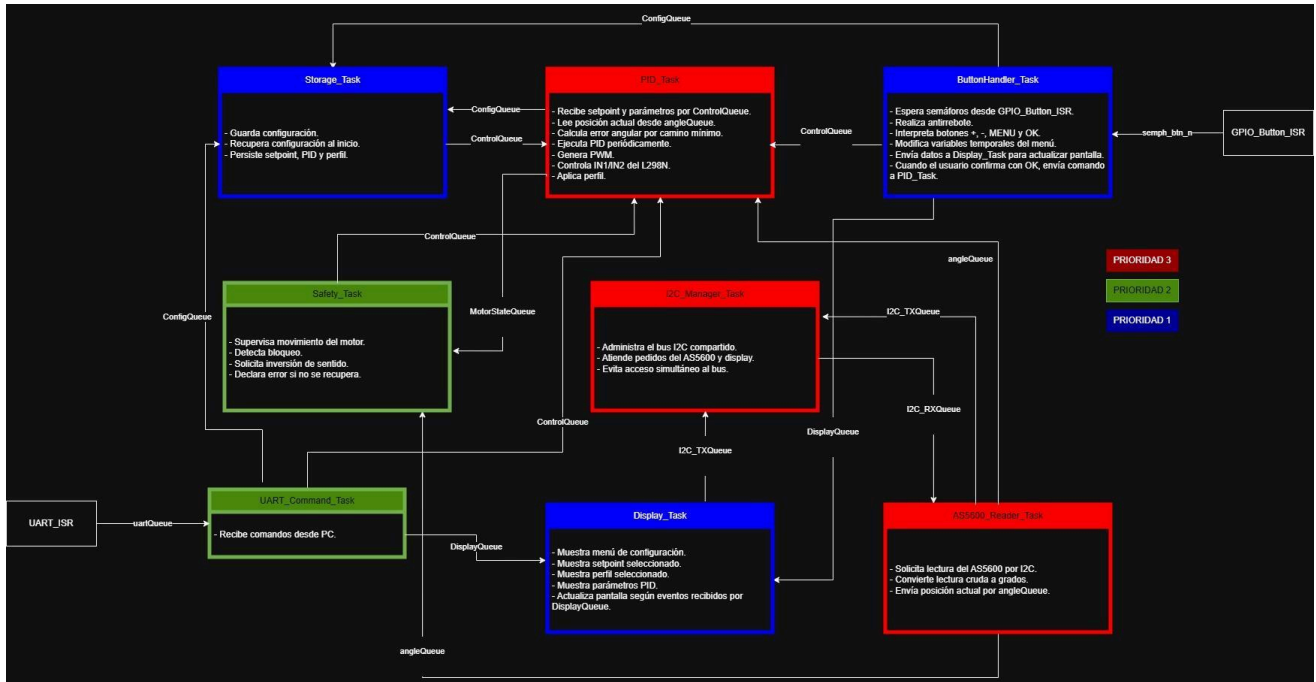


Fig. A1. Diagrama de tareas, prioridades y comunicación entre recursos del RTOS.

ANEXO II - ESQUEMÁTICO

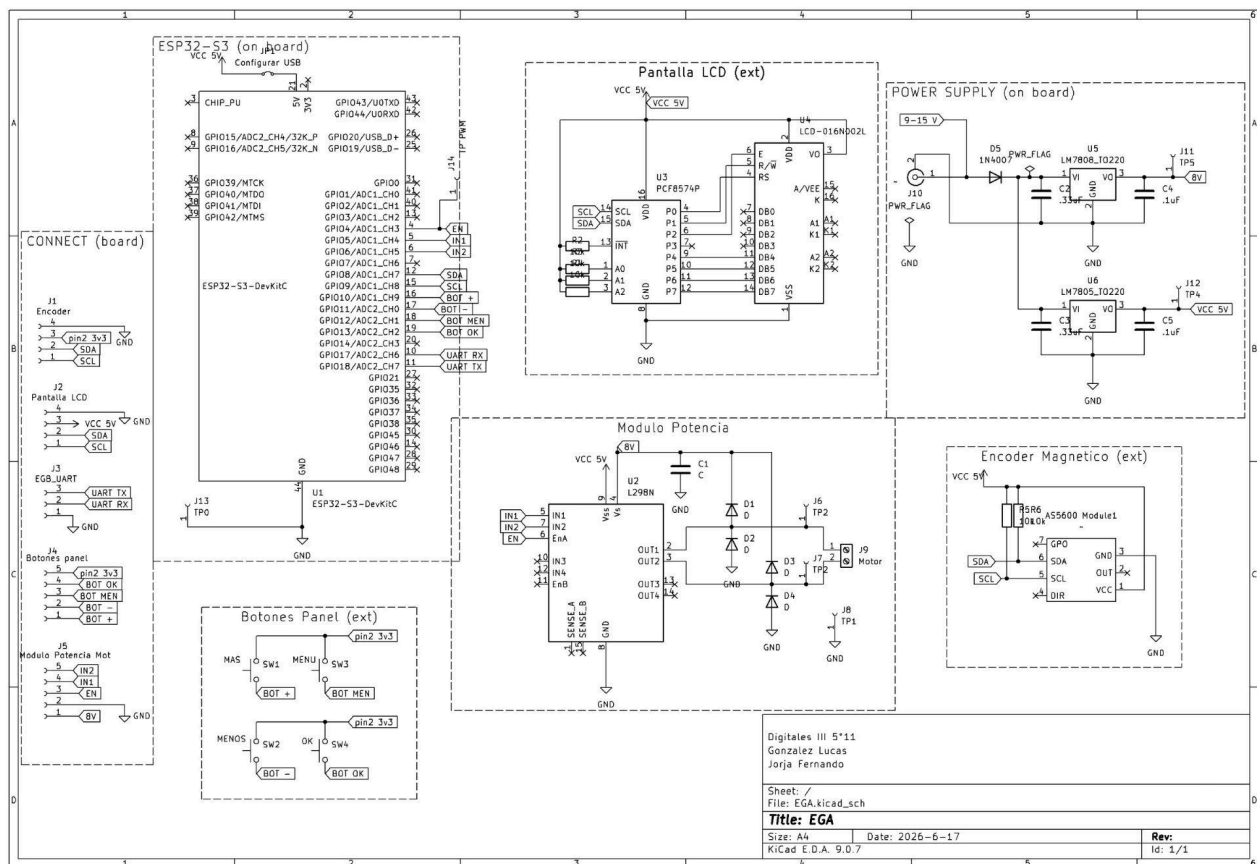


Fig. A2. Esquemático completo del sistema desarrollado en KiCad.

ANEXO III - PCB

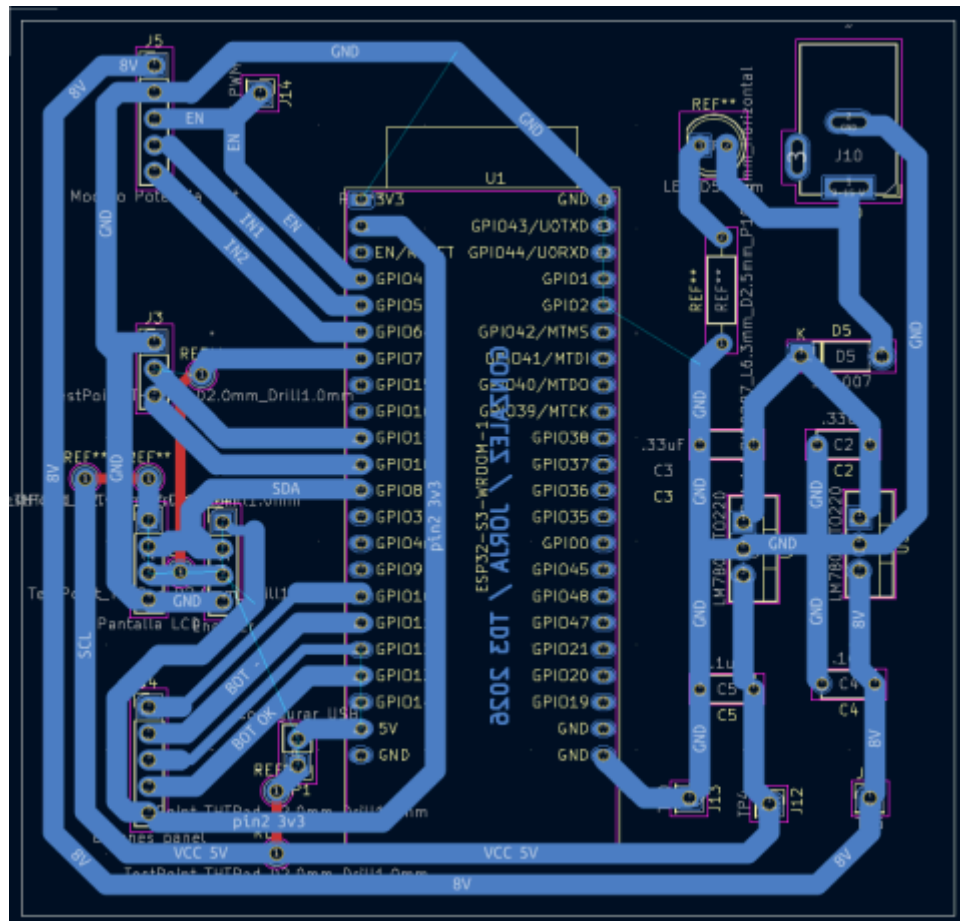


Fig. A3. Diseño del PCB del prototipo.